

CORPORACION UNIVERSITARIA MINUTO DE DIOS

Asignatura:

Estándares y Métricas de Calidad en el Desarrollo de Software

Nombre del tema:

Modelado Digital 3D

Semana 4

Presenta:

Nataly Garavito Garzón

Docente:

Cesar Alfonso Bolado Silva

Bogotá, Colombia

Julio, 26 de 2026

INTRODUCCIÓN

El aseguramiento de la calidad del software es un proceso esencial para garantizar que las aplicaciones cumplan con los requisitos establecidos, respondan a las necesidades de los usuarios y satisfagan estándares de calidad relacionados con la funcionalidad, confiabilidad, seguridad y eficiencia. Implementar prácticas de calidad desde las primeras etapas del desarrollo permite reducir errores, optimizar recursos y mejorar la calidad del producto final.

En este trabajo se presenta el modelado de un Sistema de Gestión de Incidencias de Calidad del Software (SGICS), utilizando diagramas UML para representar su estructura y funcionamiento. Además, se definen los requisitos funcionales y no funcionales, los criterios de aceptación, la trazabilidad entre requisitos y casos de prueba, y los puntos de control de calidad durante el ciclo de vida del software.

Este modelado permite evidenciar cómo una adecuada especificación del sistema facilita la organización del proceso de desarrollo, mejora la comunicación entre los participantes y contribuye a la construcción de soluciones de software más confiables y alineadas con las necesidades del cliente.

DESARROLLO

1. Nombre

Sistema de Gestión de Incidencias de Calidad del Software (SGICS)

2. Objetivo

Administrar los requisitos, los casos de prueba y las incidencias de un proyecto de software, garantizando el aseguramiento de la calidad mediante controles durante el ciclo de vida del desarrollo.

3. Descripción

El sistema propuesto permite gestionar proyectos de software, administrar requisitos, crear casos de prueba y registrar incidencias detectadas durante la validación. Asimismo, facilita la trazabilidad entre los requisitos y las pruebas realizadas, permitiendo verificar el cumplimiento de los objetivos de calidad.

4. Requisitos funcionales

Código	Requisito
RF-01	Registrar proyectos de software
RF-02	Registrar requisitos funcionales y no funcionales
RF-03	Crear casos de prueba asociados a cada requisito
RF-04	Registrar incidencias encontradas durante las pruebas
RF-05	Asignar incidencias a un desarrollador
RF-06	Registrar el estado de las pruebas
RF-07	Generar reportes de calidad del proyecto
RF-08	Consultar trazabilidad entre requisitos y pruebas

5. Requisitos no funcionales

Código	Requisito
RNF-01	Disponibilidad del 99 %
RNF-02	Tiempo de respuesta menor a 3 segundos
RNF-03	Autenticación mediante usuario y contraseña
RNF-04	Información almacenada de forma segura
RNF-05	Interfaz intuitiva y fácil de usar

6. Criterios de aceptación

Código	Criterio
RF-01	El sistema permite registrar un proyecto correctamente
RF-03	Cada caso de prueba queda asociado a un requisito.
RF-04	Las incidencias almacenan prioridad y estado
RF-06	El usuario puede actualizar el estado de una prueba.
RF-07	El sistema genera reportes sin errores.

PLANEACIÓN

Durante el desarrollo del modelo se siguieron las siguientes etapas:

- Identificación del problema
- Definición de requisitos
- Elaboración del diagrama de casos de uso
- Elaboración del diagrama de clases
- Definición de criterios de aceptación
- Diseño de la matriz de trazabilidad
- Revisión del modelo

Herramientas

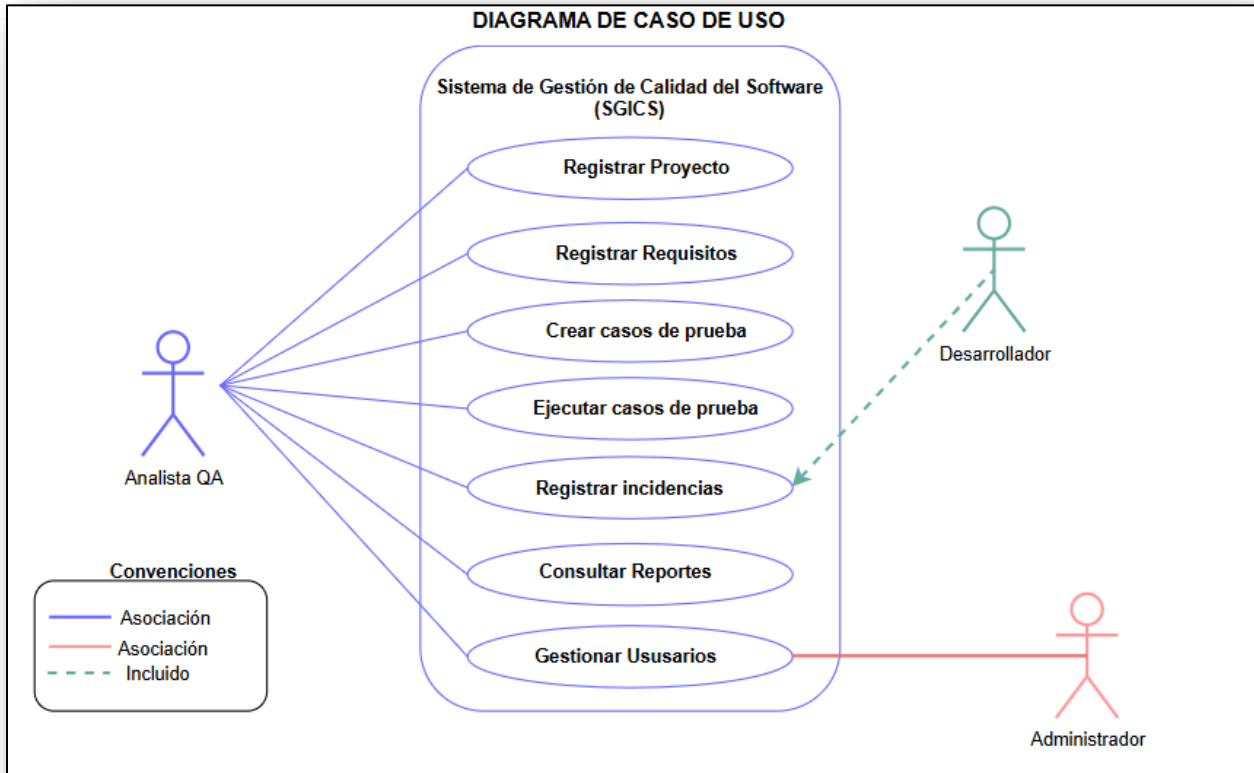
- Draw.io
- Microsoft Word

Técnicas

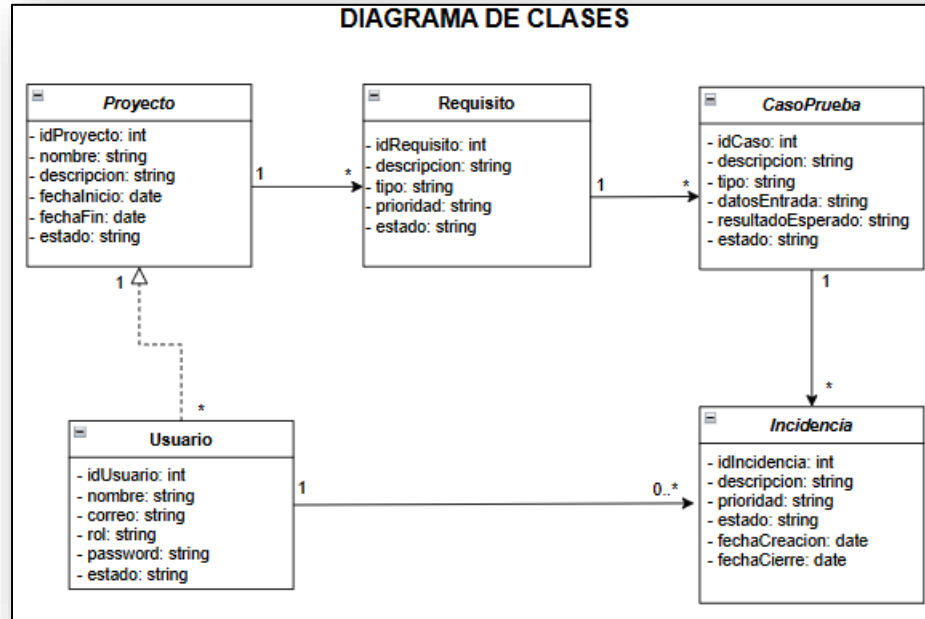
- UML
- Ingeniería de requisitos
- Aseguramiento de calidad del software

MODELADO

- Diagrama de Casos de Uso



- Diagrama de Clases



- Matriz de Trazabilidad: Requisitos – Casos de Prueba

ID Requisito	Tipo	Descripción del Requisito	ID Caso de Prueba	Descripción del Caso de Prueba	Prioridad	Estado
RF-01	Funcional	El sistema debe permitir registrar proyectos.	CP-001	Verificar registro correcto de un proyecto.	Alta	Aprobado
RF-02	Funcional	El sistema debe permitir registrar requisitos.	CP-002	Verificar creación de requisito asociado al proyecto.	Alta	Aprobado
RF-03	Funcional	El sistema debe permitir crear casos de prueba para cada requisito.	CP-003	Verificar creación de un caso de prueba asociado a un requisito.	Alta	Pendiente
RF-04	Funcional	El sistema debe permitir registrar incidencias.	CP-004	Verificar registro de incidencia con prioridad y estado.	Alta	Aprobado
RF-05	Funcional	El sistema debe permitir asignar incidencias.	CP-005	Verificar asignación de incidencia a un usuario.	Media	Pendiente
RF-06	Funcional	El sistema debe permitir registrar el estado de las pruebas.	CP-006	Verificar actualización del estado de una prueba.	Media	Aprobado

- Controles de calidad



- Justificación de las decisiones de diseño e integración del aseguramiento de la calidad

El diseño del sistema se orientó a facilitar la gestión del proceso de aseguramiento de la calidad del software mediante una estructura clara y organizada. Se definieron entidades como Proyecto, Requisito, Caso de Prueba, Incidencia y Usuario, ya que representan los elementos fundamentales para administrar un proyecto de desarrollo y realizar el seguimiento de las actividades de calidad. La relación entre estas entidades permite mantener una adecuada organización de la información y garantizar la trazabilidad de los procesos.

El diagrama de casos de uso fue diseñado para representar las principales funcionalidades del sistema desde la perspectiva de los diferentes actores, como el analista de calidad, el desarrollador y el administrador. Esta distribución de responsabilidades facilita la comprensión del funcionamiento del sistema y contribuye a una mejor definición de los procesos.

El modelo integra mecanismos de aseguramiento de la calidad mediante la definición de requisitos funcionales y no funcionales, los cuales establecen las características que debe cumplir el sistema tanto en su funcionamiento como en aspectos de rendimiento, seguridad y disponibilidad. Asimismo, se incorporan criterios de aceptación para verificar que cada requisito se cumpla correctamente antes de su aprobación.

Como complemento, se incluye una matriz de trazabilidad que relaciona cada requisito con su respectivo caso de prueba, permitiendo comprobar que todas las funcionalidades sean validadas durante la etapa de pruebas. Finalmente, se establecen puntos de control de calidad en las diferentes fases del ciclo de vida del software, como la validación de requisitos, la revisión del diseño, la revisión de código, las pruebas funcionales y las pruebas de aceptación, garantizando un seguimiento continuo de la calidad del producto desde su análisis hasta su mantenimiento.

CONCLUSIONES

- ✓ El modelado UML facilita la comprensión de la estructura y funcionamiento del sistema antes de su implementación.
- ✓ La incorporación de mecanismos de aseguramiento de la calidad desde la etapa de análisis contribuye a reducir errores y mejorar la confiabilidad del software.
- ✓ La trazabilidad entre requisitos, casos de prueba y criterios de aceptación permite verificar el cumplimiento de los objetivos funcionales y no funcionales del sistema.

REFERENCIAS

- ✓ Pressman, Roger S. Software Engineering: A Practitioner's Approach (2020). *Software Engineering: A Practitioner's Approach* (9.^a ed.). McGraw-Hill.
- ✓ Sommerville, Ian. Software Engineering (2016). *Software Engineering* (10.^a ed.). Pearson.
- ✓ ISO/IEC 25010. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*.